

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)*

Activity Tool : Scenario-Based Task (Medium)

Assessment Tool Weightage: 20%

QUESTIONS		
Q.No	Question	Bloom's Level
1	A university stores student and course data in a single table. Analyze the structure and identify normalization issues, then redesign it up to 3NF	. BL4 – Analyze
2	A company database contains employee and department details. Design appropriate SQL queries to retrieve employees working in multiple departments and justify your approach	. BL6 – Create
3	Given a relation with multiple functional dependencies, determine the candidate keys and check whether the relation satisfies BCNF. If not, decompose it.	BL5 – Evaluate
4	A banking system requires secure data access. Design views and constraints to ensure data security and integrity for different types of users.	BL6 – Create
5	A sales database contains redundant data leading to anomalies. Analyze the schema and apply normalization up to 3NF or BCNF to eliminate redundancy.	BL4 – Analyze
6	Given a dataset of customers and orders, write SQL queries using joins and subqueries to retrieve meaningful information such as frequent customers.	BL3 – Apply
7	A relation contains multi-valued dependencies. Analyze the structure and decompose it into 4NF, explaining the reasoning.	BL4 – Analyze

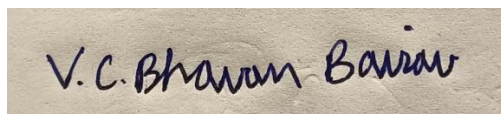
8	A database designer used improper constraints, leading to inconsistent data. Evaluate the schema and suggest appropriate integrity constraints.	BL5 – Evaluate
9	For a given relational schema, analyze the presence of join dependencies and decompose it into 5NF to remove redundancy.	BL6 – Create
10	A system uses inefficient SQL queries for data retrieval. Analyze the queries and optimize them using joins, indexing concepts, or query restructuring.	BL5 – Evaluate

Code of Conduct:

I V.C.BHAVAN BAIRAV (192511369) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

1) A university stores student and course data in a single table. Analyze the structure and identify normalization issues, then redesign it up to 3NF.

ORIGINAL TABLE:

```
mysql> SELECT * FROM Student_Course;
```

StudentID	StudentName	CourseID	CourseName	FacultyID	FacultyName
101	Aryan	C101	DBMS	F101	Dr. Kumar
101	Aryan	C102	Java	F102	Dr. Meena
102	Rahul	C101	DBMS	F101	Dr. Kumar

3 rows in set (0.00 sec)

SCENARIO UNDERSTANDING:

A university stores student, course, and faculty details in a single table. This results in repeated data, causing data redundancy and update, insert, and delete anomalies. To improve data consistency and eliminate redundancy, the database must be analyzed and normalized up to Third Normal Form (3NF).

NORMALIZATION ISSUES:

- Student details are repeated for every course.
- Faculty information is repeated for students enrolled in the same course.
- Causes **Update Anomaly**, **Insert Anomaly**, and **Delete Anomaly**.
- Contains partial and transitive dependencies.

SOLUTION APPROACH:

The database is divided into separate tables such as Student, Department, Course, and Enrollment. This removes redundancy and improves data consistency.

```

mysql> use ssp;
Database changed
mysql> CREATE TABLE Student (
->     StudentID INT PRIMARY KEY,
->     StudentName VARCHAR(50)
-> );
Query OK, 0 rows affected (0.10 sec)

mysql> CREATE TABLE Faculty (
->     FacultyID VARCHAR(10) PRIMARY KEY,
->     FacultyName VARCHAR(50)
-> );
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE TABLE Course (
->     CourseID VARCHAR(10) PRIMARY KEY,
->     CourseName VARCHAR(50),
->     FacultyID VARCHAR(10),
->     FOREIGN KEY (FacultyID) REFERENCES Faculty(FacultyID)
-> );
Query OK, 0 rows affected (0.06 sec)

mysql> CREATE TABLE Enrollment (
->     StudentID INT,
->     CourseID VARCHAR(10),
->     PRIMARY KEY (StudentID, CourseID),
->     FOREIGN KEY (StudentID) REFERENCES Student(StudentID),
->     FOREIGN KEY (CourseID) REFERENCES Course(CourseID)
-> );
Query OK, 0 rows affected (0.05 sec)

```

```

mysql> INSERT INTO Student VALUES
-> (101,'Aryan'),
-> (102,'Rahul');
Query OK, 2 rows affected (0.02 sec)
Records: 2 Duplicates: 0 Warnings: 0

mysql>
mysql> INSERT INTO Faculty VALUES
-> ('F101','Dr. Kumar'),
-> ('F102','Dr. Meena');
Query OK, 2 rows affected (0.01 sec)
Records: 2 Duplicates: 0 Warnings: 0

mysql>
mysql> INSERT INTO Course VALUES
-> ('C101','DBMS','F101'),
-> ('C102','Java','F102');
Query OK, 2 rows affected (0.01 sec)
Records: 2 Duplicates: 0 Warnings: 0

mysql>
mysql> INSERT INTO Enrollment VALUES
-> (101,'C101'),
-> (101,'C102'),
-> (102,'C101');
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0

```

```
mysql> SELECT * FROM Student;
+-----+-----+
| StudentID | StudentName |
+-----+-----+
|      101 | Aryan      |
|      102 | Rahul      |
+-----+-----+
2 rows in set (0.00 sec)

mysql> SELECT * FROM Faculty;
+-----+-----+
| FacultyID | FacultyName |
+-----+-----+
| F101      | Dr. Kumar   |
| F102      | Dr. Meena   |
+-----+-----+
2 rows in set (0.00 sec)

mysql> SELECT * FROM Course;
+-----+-----+-----+
| CourseID | CourseName | FacultyID |
+-----+-----+-----+
| C101     | DBMS       | F101      |
| C102     | Java       | F102      |
+-----+-----+-----+
2 rows in set (0.00 sec)

mysql> SELECT * FROM Enrollment;
+-----+-----+
| StudentID | CourseID |
+-----+-----+
|      101 | C101     |
|      102 | C101     |
|      101 | C102     |
+-----+-----+
3 rows in set (0.00 sec)

mysql>
```

RESULT AND JUSTIFICATION:

The database is successfully normalized up to Third Normal Form (3NF) by separating student, faculty, course, and enrollment information into individual tables. This reduces data redundancy, removes update, insert, and delete anomalies, and improves data integrity and maintainability.

2) A company database contains employee and department details. Design appropriate SQL queries to retrieve employees working in multiple departments and justify your approach.

SCENARIO UNDERSTANDING:

A company maintains employee and department information in its database. Some employees are assigned to work in more than one department based on project requirements. The database should accurately represent this many-to-many relationship and retrieve employees working across multiple departments efficiently.

SOLUTION APPROACH:

The database is designed using Employee, Department, and Employee_Department tables. A junction table is used to establish the many-to-many relationship between employees and departments. SQL JOIN, GROUP BY, and HAVING clauses are used to retrieve employees working in multiple departments.

```
mysql> CREATE TABLE Employee (  
->     Emp_ID INT PRIMARY KEY,  
->     Emp_Name VARCHAR(50),  
->     Designation VARCHAR(50)  
-> );  
Query OK, 0 rows affected (0.06 sec)  
  
mysql>  
mysql> INSERT INTO Employee VALUES  
-> (101, 'Arun', 'Manager'),  
-> (102, 'Priya', 'Developer'),  
-> (103, 'Rahul', 'Analyst'),  
-> (104, 'Meena', 'Tester');  
Query OK, 4 rows affected (0.01 sec)  
Records: 4 Duplicates: 0 Warnings: 0  
  
mysql>  
mysql> CREATE TABLE Department (  
->     Dept_ID INT PRIMARY KEY,  
->     Dept_Name VARCHAR(50)  
-> );  
Query OK, 0 rows affected (0.04 sec)  
  
mysql>  
mysql> INSERT INTO Department VALUES  
-> (1, 'HR'),  
-> (2, 'IT'),  
-> (3, 'Finance');  
Query OK, 3 rows affected (0.01 sec)  
Records: 3 Duplicates: 0 Warnings: 0
```

```

mysql>
mysql> CREATE TABLE Employee_Department (
  ->     Emp_ID INT,
  ->     Dept_ID INT,
  ->     PRIMARY KEY (Emp_ID, Dept_ID),
  ->     FOREIGN KEY (Emp_ID) REFERENCES Employee(Emp_ID),
  ->     FOREIGN KEY (Dept_ID) REFERENCES Department(Dept_ID)
  -> );
Query OK, 0 rows affected (0.10 sec)

mysql>
mysql> INSERT INTO Employee_Department VALUES
  -> (101,1),
  -> (101,2),
  -> (102,2),
  -> (103,2),
  -> (103,3),
  -> (104,1);
Query OK, 6 rows affected (0.01 sec)
Records: 6 Duplicates: 0 Warnings: 0

mysql>
mysql> SELECT
  ->     E.Emp_ID,
  ->     E.Emp_Name,
  ->     COUNT(ED.Dept_ID) AS Total_Departments
  -> FROM Employee E
  -> INNER JOIN Employee_Department ED
  -> ON E.Emp_ID = ED.Emp_ID
  -> GROUP BY E.Emp_ID, E.Emp_Name
  -> HAVING COUNT(ED.Dept_ID) > 1;
+-----+-----+-----+
| Emp_ID | Emp_Name | Total_Departments |
+-----+-----+-----+
| 101    | Arun    | 2                 |
| 103    | Rahul   | 2                 |
+-----+-----+-----+
2 rows in set (0.00 sec)

```

RESULT AND JUSTIFICATION:

The SQL query successfully identifies employees working in multiple departments by combining data from related tables using JOIN and filtering the results with GROUP BY and HAVING. This approach accurately represents the many-to-many relationship, avoids data redundancy, and provides efficient retrieval of employee information for organizational analysis.

3) Given a relation with multiple functional dependencies, determine the candidate keys and check whether the relation satisfies BCNF. If not, decompose it.

ORIGINAL TABLE:

```
mysql> SELECT * FROM Enrollment;
```

StudentID	CourseID	Instructor	InstructorPhone
101	C101	Dr. Kumar	9876543210
101	C102	Dr. Meena	9876543220
102	C101	Dr. Kumar	9876543210
103	C102	Dr. Meena	9876543220

4 rows in set (0.00 sec)

SCENARIO UNDERSTANDING:

The given relation stores student, course, and instructor details in a single table. Since instructor information is repeated for multiple student-course combinations, the relation contains functional dependencies that may lead to redundancy and update anomalies. Therefore, the relation must be analyzed to determine its candidate keys and checked for Boyce-Codd Normal Form (BCNF).

The relation **does not satisfy BCNF** because:

- $\text{Instructor} \rightarrow \text{InstructorPhone}$
- **Instructor** is **not a candidate key**.
- Therefore, the relation violates **BCNF**.

SOLUTION APPROACH:

The functional dependencies are analyzed to identify the candidate key and determine whether the relation satisfies BCNF. Since the dependency $\text{Instructor} \rightarrow \text{InstructorPhone}$ violates BCNF, the relation is decomposed into separate Instructor and Enrollment tables. This decomposition removes redundancy while preserving the relationship between students, courses, and instructors.

MySQL 8.0 Command Line Client

```
mysql> CREATE TABLE Instructor (  
->     Instructor VARCHAR(50) PRIMARY KEY,  
->     InstructorPhone BIGINT  
-> );
```

Query OK, 0 rows affected (0.03 sec)

```
mysql> INSERT INTO Instructor VALUES  
-> ('Dr. Kumar',9876543210),  
-> ('Dr. Meena',9876543220);
```

Query OK, 2 rows affected (0.01 sec)

Records: 2 Duplicates: 0 Warnings: 0

```
mysql> CREATE TABLE Enrollment_BCNF (  
->     StudentID INT,  
->     CourseID VARCHAR(10),  
->     Instructor VARCHAR(50),  
->     PRIMARY KEY (StudentID, CourseID),  
->     FOREIGN KEY (Instructor)  
->     REFERENCES Instructor(Instructor)  
-> );
```

Query OK, 0 rows affected (0.05 sec)

```
mysql> INSERT INTO Enrollment_BCNF VALUES  
-> (101,'C101','Dr. Kumar'),  
-> (101,'C102','Dr. Meena'),  
-> (102,'C101','Dr. Kumar'),  
-> (103,'C102','Dr. Meena');
```

Query OK, 4 rows affected (0.01 sec)

Records: 4 Duplicates: 0 Warnings: 0

```
mysql> SELECT * FROM Instructor;
```

Instructor	InstructorPhone
Dr. Kumar	9876543210
Dr. Meena	9876543220

2 rows in set (0.00 sec)

```
mysql>
```

```
mysql> SELECT * FROM Enrollment_BCNF;
```

StudentID	CourseID	Instructor
101	C101	Dr. Kumar
102	C101	Dr. Kumar
101	C102	Dr. Meena
103	C102	Dr. Meena

4 rows in set (0.00 sec)

RESULT AND JUSTIFICATION:

The relation was successfully decomposed into BCNF by separating instructor details from enrollment information. The decomposition removes redundancy, eliminates update anomalies, and improves data integrity while preserving all functional dependencies and relationships between the data.

4) A banking system requires secure data access. Design views and constraints to ensure data security and integrity for different types of users.

SCENARIO UNDERSTANDING:

A banking system stores customer account information such as account number, customer name, balance, and PIN. Different users require different levels of access to the data. To ensure data security and integrity, Views are used to restrict sensitive information, while Constraints are applied to maintain valid and consistent data.

SOLUTION APPROACH:

A BankAccount table is created with constraints such as PRIMARY KEY, NOT NULL, CHECK, and UNIQUE. A View is created to display only non-sensitive information to general users while hiding confidential details like the PIN.

```
mysql> CREATE TABLE BankAccount (  
->     AccountNo INT PRIMARY KEY,  
->     CustomerName VARCHAR(50) NOT NULL,  
->     Balance DECIMAL(10,2) CHECK (Balance >= 0),  
->     PIN INT UNIQUE  
-> );  
Query OK, 0 rows affected (0.06 sec)  
  
mysql> INSERT INTO BankAccount VALUES  
-> (1001, 'Arun', 25000, 1234),  
-> (1002, 'Priya', 18000, 5678),  
-> (1003, 'Rahul', 32000, 9876);  
Query OK, 3 rows affected (0.01 sec)  
Records: 3  Duplicates: 0  Warnings: 0
```

```
mysql> CREATE VIEW Customer_View AS
-> SELECT AccountNo,
->         CustomerName,
->         Balance
-> FROM BankAccount;
Query OK, 0 rows affected (0.01 sec)

mysql> SELECT * FROM Customer_View;
+-----+-----+-----+
| AccountNo | CustomerName | Balance |
+-----+-----+-----+
|      1001 | Arun         | 25000.00 |
|      1002 | Priya        | 18000.00 |
|      1003 | Rahul        | 32000.00 |
+-----+-----+-----+
3 rows in set (0.00 sec)
```

RESULT AND JUSTIFICATION:

The banking database was successfully secured by implementing views and integrity constraints. The Customer_View provides restricted access by displaying only non-sensitive information, while sensitive data such as the PIN remains hidden. The use of PRIMARY KEY, NOT NULL, UNIQUE, and CHECK constraints ensures data accuracy, prevents invalid entries, and maintains database integrity. This design enhances both security and consistency, making the system suitable for different types of users.

5) A sales database contains redundant data leading to anomalies. Analyze the schema and apply normalization up to 3NF or BCNF to eliminate redundancy.

SCENARIO UNDERSTANDING:

A sales database stores customer, product, and sales information in a single table. This results in repeated customer and product details, causing data redundancy and anomalies during insertion, updation, and deletion of records. To improve data consistency and eliminate redundancy, the database is normalized up to Third Normal Form (3NF).

SOLUTION APPROACH:

The database is decomposed into separate Customer, Product, and Sales tables. Primary and Foreign Keys are used to establish relationships between the tables, thereby removing redundancy and ensuring data integrity.

```
mysql> CREATE TABLE Customer (
->     CustomerID INT PRIMARY KEY,
->     CustomerName VARCHAR(50)
-> );
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE TABLE Product (
->     ProductID INT PRIMARY KEY,
->     ProductName VARCHAR(50),
->     Price DECIMAL(10,2)
-> );
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE TABLE Sales (
->     SaleID INT PRIMARY KEY,
->     CustomerID INT,
->     ProductID INT,
->     Quantity INT,
->     FOREIGN KEY (CustomerID) REFERENCES Customer(CustomerID),
->     FOREIGN KEY (ProductID) REFERENCES Product(ProductID)
-> );
Query OK, 0 rows affected (0.06 sec)

mysql> INSERT INTO Customer VALUES
-> (101,'Arun'),
-> (102,'Priya');
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Product VALUES
-> (201,'Laptop',55000),
-> (202,'Mouse',800);
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Sales VALUES
-> (1,101,201,1),
-> (2,101,202,2),
-> (3,102,201,1);
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

```
mysql> SELECT * FROM Customer;
+-----+-----+
| CustomerID | CustomerName |
+-----+-----+
|          101 | Arun          |
|          102 | Priya         |
+-----+-----+
2 rows in set (0.00 sec)

mysql> SELECT * FROM Product;
+-----+-----+-----+
| ProductID | ProductName | Price |
+-----+-----+-----+
|          201 | Laptop      | 55000.00 |
|          202 | Mouse       | 800.00 |
+-----+-----+-----+
2 rows in set (0.00 sec)

mysql> SELECT * FROM Sales;
+-----+-----+-----+-----+
| SaleID | CustomerID | ProductID | Quantity |
+-----+-----+-----+-----+
|        1 |          101 |          201 |          1 |
|        2 |          101 |          202 |          2 |
|        3 |          102 |          201 |          1 |
+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

RESULT AND JUSTIFICATION:

The sales database was successfully normalized up to Third Normal Form (3NF) by separating customer, product, and sales information into individual tables. This eliminates data redundancy, removes update, insert, and delete anomalies, and maintains relationships using primary and foreign keys. The normalized design improves data consistency, integrity, and overall database efficiency.

6) Given a dataset of customers and orders, write SQL queries using joins and subqueries to retrieve meaningful information such as frequent customers.

SCENARIO UNDERSTANDING:

A company maintains separate tables for Customers and Orders. Each customer can place multiple orders. The objective is to identify frequent customers using JOIN and Subquery operations to retrieve meaningful business information.

SOLUTION APPROACH:

The database is designed using Customer and Orders tables. A JOIN combines customer and order details, while a Subquery identifies customers who have placed more than one order.

```
MySQL 8.0 Command Line Client
Database changed
mysql> CREATE TABLE Customer (
  ->     CustomerID INT PRIMARY KEY,
  ->     CustomerName VARCHAR(50)
  -> );
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE TABLE Orders (
  ->     OrderID INT PRIMARY KEY,
  ->     CustomerID INT,
  ->     Amount DECIMAL(10,2),
  ->     FOREIGN KEY (CustomerID) REFERENCES Customer(CustomerID)
  -> );
Query OK, 0 rows affected (0.04 sec)

mysql> INSERT INTO Customer VALUES
  -> (101,'Arun'),
  -> (102,'Priya'),
  -> (103,'Rahul');
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql>
mysql> INSERT INTO Orders VALUES
  -> (1,101,2500),
  -> (2,101,1800),
  -> (3,102,3000),
  -> (4,103,1200),
  -> (5,103,1500);
Query OK, 5 rows affected (0.01 sec)
Records: 5 Duplicates: 0 Warnings: 0
```

```

mysql> SELECT C.CustomerName,
->         O.OrderID,
->         O.Amount
-> FROM Customer C
-> JOIN Orders O
-> ON C.CustomerID = O.CustomerID;
+-----+-----+-----+
| CustomerName | OrderID | Amount |
+-----+-----+-----+
| Arun         | 1       | 2500.00 |
| Arun         | 2       | 1800.00 |
| Priya        | 3       | 3000.00 |
| Rahul        | 4       | 1200.00 |
| Rahul        | 5       | 1500.00 |
+-----+-----+-----+
5 rows in set (0.00 sec)

mysql> SELECT CustomerName
-> FROM Customer
-> WHERE CustomerID IN
-> (
->     SELECT CustomerID
->     FROM Orders
->     GROUP BY CustomerID
->     HAVING COUNT(OrderID) > 1
-> );
+-----+
| CustomerName |
+-----+
| Arun         |
| Rahul        |
+-----+
2 rows in set (0.00 sec)

```

RESULT AND JUSTIFICATION:

The required information was successfully retrieved using JOIN and Subquery operations. The JOIN combines customer and order details, while the subquery identifies customers who have placed multiple orders. This approach efficiently retrieves meaningful business information, reduces query complexity, and improves database analysis.

7) A relation contains multi-valued dependencies. Analyze the structure and decompose it into 4NF, explaining the reasoning.

SCENARIO UNDERSTANDING:

A relation contains multi-valued dependencies, where one attribute is associated with multiple independent values. This results in redundant data and unnecessary duplication. To eliminate redundancy and improve data consistency, the relation is decomposed into Fourth Normal Form (4NF).

SOLUTION APPROACH:

The relation is analyzed to identify the multi-valued dependencies. The table is then decomposed into separate relations so that each table contains only one independent multi-valued attribute, thereby satisfying 4NF.

```
MySQL 8.0 Command Line Client
mysql> CREATE TABLE Student_Course (
->   StudentID INT,
->   Course VARCHAR(50),
->   PRIMARY KEY (StudentID, Course)
-> );
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE TABLE Student_Hobby (
->   StudentID INT,
->   Hobby VARCHAR(50),
->   PRIMARY KEY (StudentID, Hobby)
-> );
Query OK, 0 rows affected (0.02 sec)

mysql> INSERT INTO Student_Course VALUES
->   (101, 'DBMS'),
->   (101, 'Java');
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Student_Hobby VALUES
->   (101, 'Cricket'),
->   (101, 'Music');
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0
```

```
mysql> SELECT * FROM Student_Course;
+-----+-----+
| StudentID | Course |
+-----+-----+
|      101 | DBMS   |
|      101 | Java   |
+-----+-----+
2 rows in set (0.00 sec)

mysql>
mysql> SELECT * FROM Student_Hobby;
+-----+-----+
| StudentID | Hobby  |
+-----+-----+
|      101 | Cricket |
|      101 | Music  |
+-----+-----+
2 rows in set (0.00 sec)
```

RESULT AND JUSTIFICATION:

The relation was successfully decomposed into Fourth Normal Form (4NF) by separating the independent multi-valued attributes into Student_Course and Student_Hobby tables. This removes redundancy, eliminates multi-valued dependencies, prevents update anomalies, and improves data consistency while preserving all required information.

8) A database designer used improper constraints, leading to inconsistent data. Evaluate the schema and suggest appropriate integrity constraints.

SCENARIO UNDERSTANDING:

A database stores customer information without proper integrity constraints. As a result, duplicate records, NULL values, and invalid data can be entered, leading to inconsistencies. To improve data integrity, appropriate constraints must be applied to the table.

SOLUTION APPROACH:

The database schema is evaluated, and integrity constraints such as PRIMARY KEY, NOT NULL, UNIQUE, CHECK, and FOREIGN KEY are applied to ensure accurate and consistent data while preventing invalid entries.

```
mysql> CREATE TABLE Customer (  
->     CustomerID INT PRIMARY KEY,  
->     CustomerName VARCHAR(50) NOT NULL,  
->     Email VARCHAR(100) UNIQUE,  
->     Age INT CHECK (Age >= 18)  
-> );  
Query OK, 0 rows affected (0.05 sec)  
  
mysql> CREATE TABLE Orders (  
->     OrderID INT PRIMARY KEY,  
->     CustomerID INT,  
->     OrderAmount DECIMAL(10,2) CHECK (OrderAmount > 0),  
->     FOREIGN KEY (CustomerID) REFERENCES Customer(CustomerID)  
-> );  
Query OK, 0 rows affected (0.06 sec)  
  
mysql> INSERT INTO Customer VALUES  
-> (101,'Arun','arun@gmail.com',25),  
-> (102,'Priya','priya@gmail.com',30);  
Query OK, 2 rows affected (0.01 sec)  
Records: 2 Duplicates: 0 Warnings: 0  
  
mysql>  
mysql> INSERT INTO Orders VALUES  
-> (1,101,2500),  
-> (2,102,1800);  
Query OK, 2 rows affected (0.01 sec)  
Records: 2 Duplicates: 0 Warnings: 0
```

```
mysql> SELECT * FROM Customer;
+-----+-----+-----+-----+
| CustomerID | CustomerName | Email | Age |
+-----+-----+-----+-----+
| 101 | Arun | arun@gmail.com | 25 |
| 102 | Priya | priya@gmail.com | 30 |
+-----+-----+-----+-----+
2 rows in set (0.00 sec)

mysql>
mysql> SELECT * FROM Orders;
+-----+-----+-----+
| OrderID | CustomerID | OrderAmount |
+-----+-----+-----+
| 1 | 101 | 2500.00 |
| 2 | 102 | 1800.00 |
+-----+-----+-----+
2 rows in set (0.00 sec)
```

RESULT AND JUSTIFICATION:

The schema was successfully improved by applying PRIMARY KEY, NOT NULL, UNIQUE, CHECK, and FOREIGN KEY constraints. These constraints prevent duplicate records, NULL values, invalid ages and order amounts, and maintain valid relationships between tables. As a result, the database becomes more secure, consistent, and reliable by ensuring data integrity and reducing inconsistencies.

9) For a given relational schema, analyze the presence of join dependencies and decompose it into 5NF to remove redundancy.

SCENARIO UNDERSTANDING:

A database stores Supplier, Product, and Project information in a single table. Since suppliers can supply multiple products to multiple projects, the relation contains join dependencies, resulting in redundant data. To eliminate redundancy and maintain consistency, the relation is decomposed into Fifth Normal Form (5NF).

SOLUTION APPROACH:

The relation is analyzed to identify the join dependency and is decomposed into smaller relations. Separate tables are created for Supplier-Product, Supplier-Project, and Product-Project relationships. This decomposition removes redundancy while preserving all original information.

```
mysql> CREATE TABLE Supplier_Product (
->     SupplierID VARCHAR(10),
->     ProductID VARCHAR(10),
->     PRIMARY KEY (SupplierID, ProductID)
-> );
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE TABLE Supplier_Project (
->     SupplierID VARCHAR(10),
->     ProjectID VARCHAR(10),
->     PRIMARY KEY (SupplierID, ProjectID)
-> );
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE TABLE Product_Project (
->     ProductID VARCHAR(10),
->     ProjectID VARCHAR(10),
->     PRIMARY KEY (ProductID, ProjectID)
-> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO Supplier_Product VALUES
->     ('S1','P1'),
->     ('S1','P2'),
->     ('S2','P1');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Supplier_Project VALUES
->     ('S1','PR1'),
->     ('S2','PR2');
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Product_Project VALUES
->     ('P1','PR1'),
->     ('P2','PR1'),
->     ('P1','PR2');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

```
MySQL 8.0 Command Line Client

mysql> SELECT * FROM Supplier_Product;
+-----+-----+
| SupplierID | ProductID |
+-----+-----+
| S1         | P1        |
| S1         | P2        |
| S2         | P1        |
+-----+-----+
3 rows in set (0.00 sec)

mysql>
mysql> SELECT * FROM Supplier_Project;
+-----+-----+
| SupplierID | ProjectID |
+-----+-----+
| S1         | PR1       |
| S2         | PR2       |
+-----+-----+
2 rows in set (0.00 sec)

mysql>
mysql> SELECT * FROM Product_Project;
+-----+-----+
| ProductID | ProjectID |
+-----+-----+
| P1        | PR1       |
| P1        | PR2       |
| P2        | PR1       |
+-----+-----+
3 rows in set (0.00 sec)
```

RESULT AND JUSTIFICATION:

The function improves query readability, supports code reuse, and provides consistent results across the database

10) A system uses inefficient SQL queries for data retrieval. Analyze the queries and optimize them using joins, indexing concepts, or query restructuring.

SCENARIO UNDERSTANDING:

A database system contains SQL queries that retrieve data inefficiently, resulting in slower execution and increased resource usage. The queries need to be analyzed and optimized using appropriate techniques such as JOINS, Indexing, and Query Restructuring to improve performance.

SOLUTION APPROACH:

The inefficient SQL queries are examined and optimized by replacing unnecessary subqueries with JOINS, creating indexes on frequently searched columns, and restructuring queries to reduce execution time and improve database performance.

```
mysql> CREATE TABLE Department (  
->     Dept_ID INT PRIMARY KEY,  
->     Dept_Name VARCHAR(50)  
-> );  
Query OK, 0 rows affected (0.07 sec)  
  
mysql>  
mysql> INSERT INTO Department VALUES  
-> (1,'HR'),  
-> (2,'IT'),  
-> (3,'Finance');  
Query OK, 3 rows affected (0.01 sec)  
Records: 3 Duplicates: 0 Warnings: 0  
  
mysql>  
mysql> CREATE TABLE Employee (  
->     Emp_ID INT PRIMARY KEY,  
->     Emp_Name VARCHAR(50),  
->     Salary DECIMAL(10,2),  
->     Dept_ID INT,  
->     FOREIGN KEY (Dept_ID) REFERENCES Department(Dept_ID)  
-> );  
Query OK, 0 rows affected (0.10 sec)  
  
mysql>  
mysql> INSERT INTO Employee VALUES  
-> (101,'Arun',30000,1),  
-> (102,'Priya',45000,2),  
-> (103,'Rahul',50000,3),  
-> (104,'Meena',40000,2),  
-> (105,'Karan',35000,1);  
Query OK, 5 rows affected (0.04 sec)  
Records: 5 Duplicates: 0 Warnings: 0
```

QUERY RESTRUCTURING:

```
mysql> SELECT Emp_Name
-> FROM Employee
-> WHERE Dept_ID =
-> (
->     SELECT Dept_ID
->     FROM Department
->     WHERE Dept_Name = 'IT'
-> );
```

Emp_Name
Priya
Meena

2 rows in set (0.00 sec)

USING JOINS:

```
mysql> SELECT
->     E.Emp_ID,
->     E.Emp_Name,
->     D.Dept_Name,
->     E.Salary
-> FROM Employee E
-> INNER JOIN Department D
-> ON E.Dept_ID = D.Dept_ID
-> WHERE D.Dept_Name = 'IT';
```

Emp_ID	Emp_Name	Dept_Name	Salary
102	Priya	IT	45000.00
104	Meena	IT	40000.00

2 rows in set (0.04 sec)

USING INDEX:

```
mysql> CREATE INDEX idx_Dept_ID
-> ON Employee(Dept_ID);
Query OK, 0 rows affected (0.07 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

```
mysql> SELECT
->     E.Emp_ID,
->     E.Emp_Name,
->     D.Dept_Name,
->     E.Salary
-> FROM Employee E
-> INNER JOIN Department D
-> ON E.Dept_ID = D.Dept_ID;
```

Emp_ID	Emp_Name	Dept_Name	Salary
101	Arun	HR	30000.00
105	Karan	HR	35000.00
102	Priya	IT	45000.00
104	Meena	IT	40000.00
103	Rahul	Finance	50000.00

5 rows in set (0.00 sec)

mysql> |

RESULT AND JUSTIFICATION:

The SQL queries were successfully optimized using JOINS, Indexing, and Query Restructuring. These optimizations reduce unnecessary operations, improve query execution speed, minimize resource usage, and enhance overall database performance while producing the same results.